

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ,
МЕХАНИКИ И ОПТИКИ»**

Факультет безопасности информационных технологий

Дисциплина:

«Основы системного программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1.1

Работа с файлами и каталогами

Вариант 5

Выполнил:

Студент гр. N3248

Жук Анастасия Валерьевна

Подпись: _____

Проверил:

Грозов Владимир Андреевич

Подпись: _____

Санкт-Петербург

2025 г.

Задание

Разработать на языке C для ОС Linux программу, позволяющую выполнять рекурсивный поиск с помощью функций `fts_open()/fts_read()/...` файлов, начиная с указанного каталога, содержащих заданную последовательность байтов.

Программа должна представлять собой консольную утилиту, настройка работы которой осуществляется путем передачи аргументов в строке запуска и/или с помощью переменных окружения (опции необязательны, аргументы каталог и `цель_поиска` – обязательны):

```
lab11abcNXXXXXX [опции] каталог цель_поиска
```

Программа должна выполнять рекурсивный поиск файлов, отвечающих критерию, который задается аргументом `цель_поиска` в соответствии с условием: аргумент `цель_поиска` имеет формат `0xhh[hh*]`, где `hh` – две шестнадцатеричных цифры (число должно начинаться с префикса `0x`). При обнаружении файла, отвечающего заданным критериям поиска, программа должна вывести в стандартный поток вывода полный путь к этому файлу.

При указании опций `-h` или `-v` (или их "длинных" аналогов `--help` или `--version`) выполняется вывод информации, заданной опцией, и работа программы завершается.

При определении переменной окружения `LAB11DEBUG` в стандартный поток ошибок должна выводиться информация о том, что и в каком месте файла нашлось (чтобы было легче понять, почему файл отвечает критериям поиска), а также может выводиться любая дополнительная отладочная информация.

Make-файл

```
1  CC = gcc
2  CFLAGS = -Wall -Wextra -Werror -std=c11 -O3
3  TARGET = lab11avzN3248
4  SOURCES = lab11avzN3248.c
5
6  all: $(TARGET)
7
8  $(TARGET): $(SOURCES)
9      $(CC) $(CFLAGS) -o $@ $^
10
11 clean:
12     rm -f $(TARGET)
13
14 .PHONY: all clean
```

Листинг 1 – Текстовая версия Make-файла

CC = gcc

CFLAGS = -Wall -Wextra -Werror -std=c11 -O3

TARGET = lab11avzN3248

SOURCES = lab11avzN3248.c

all: \$(TARGET)

\$(TARGET): \$(SOURCES)

\$(CC) \$(CFLAGS) -o \$@ \$^

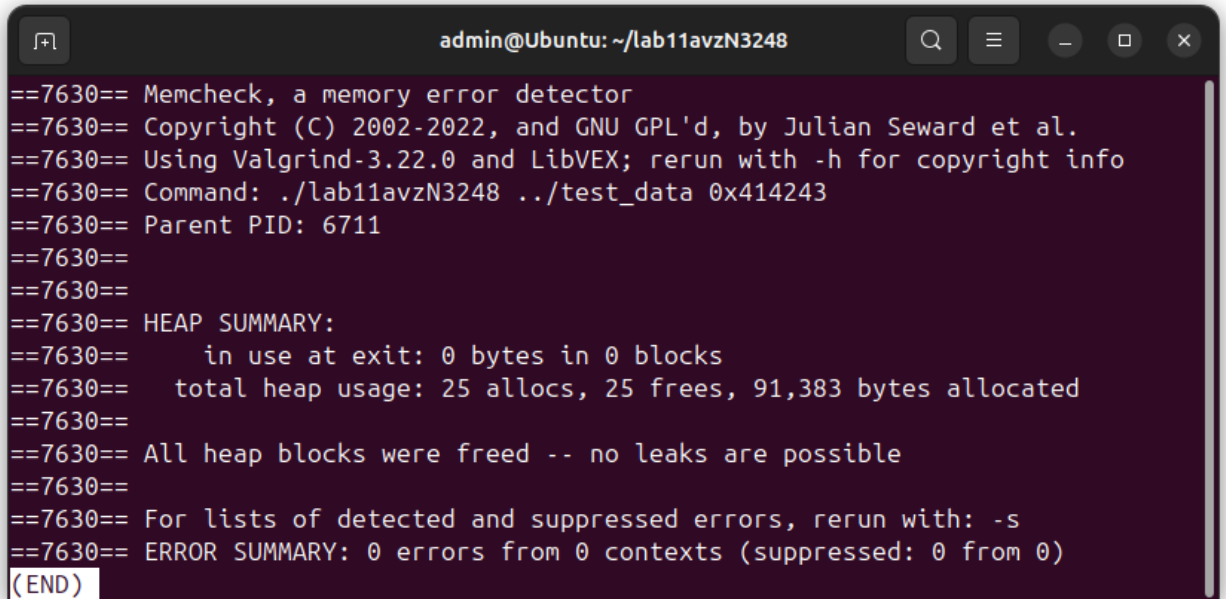
clean:

rm -f \$(TARGET)

.PHONY: all clean

Отчет valgrind

```
admin@Ubuntu:~$ cd ./lab11avzN3248
admin@Ubuntu:~/lab11avzN3248$ make all
gcc -g -Wall -Wextra -Werror -std=c11 -O3 -o lab11avzN3248 lab11avzN3248.c
admin@Ubuntu:~/lab11avzN3248$ valgrind --leak-check=full --show-leak-kinds=all -
-track-origins=yes --log-file=valgrind.txt ./lab11avzN3248 ../test_data 0x414243
../test_data/text.txt
../test_data/exact_match.bin
../test_data/middle_match.bin
admin@Ubuntu:~/lab11avzN3248$ less valgrind.txt
```



```
admin@Ubuntu: ~/lab11avzN3248
==7630== Memcheck, a memory error detector
==7630== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==7630== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==7630== Command: ./lab11avzN3248 ../test_data 0x414243
==7630== Parent PID: 6711
==7630==
==7630==
==7630== HEAP SUMMARY:
==7630==   in use at exit: 0 bytes in 0 blocks
==7630== total heap usage: 25 allocs, 25 frees, 91,383 bytes allocated
==7630==
==7630== All heap blocks were freed -- no leaks are possible
==7630==
==7630== For lists of detected and suppressed errors, rerun with: -s
==7630== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
(END)
```

==7630== HEAP SUMMARY:

==7630== in use at exit: 0 bytes in 0 blocks

==7630== total heap usage: 25 allocs, 25 frees, 91,383 bytes allocated

==7630==

==7630== All heap blocks were freed -- no leaks are possible

==7630==

==7630== For lists of detected and suppressed errors, rerun with: -s

==7630== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

Примеры работы программы

```
admin@Ubuntu:~/lab11avzN3248$ make all
gcc -Wall -Wextra -Werror -std=c11 -O3 -o lab11avzN3248 lab11avzN3248.c
admin@Ubuntu:~/lab11avzN3248$ ./lab_1.1_OSP ~/test_data 0x414243
bash: ./lab_1.1_OSP: No such file or directory
admin@Ubuntu:~/lab11avzN3248$ make clean
rm -f lab11avzN3248
admin@Ubuntu:~/lab11avzN3248$ make all
gcc -Wall -Wextra -Werror -std=c11 -O3 -o lab11avzN3248 lab11avzN3248.c
admin@Ubuntu:~/lab11avzN3248$ ./lab11avzN3248 ~/test_data 0x414243
/home/admin/test_data/text.txt
/home/admin/test_data/exact_match.bin
/home/admin/test_data/middle_match.bin
admin@Ubuntu:~/lab11avzN3248$ ./lab11avzN3248 ../test_data 0x414243
../test_data/text.txt
../test_data/exact_match.bin
../test_data/middle_match.bin
admin@Ubuntu:~/lab11avzN3248$ ./lab11avzN3248 -v ../test_data 0x414243
ОСП ЛР1.1
Версия: 1.0
Автор: Жук Анастасия Валерьевна
Группа: N3248
admin@Ubuntu:~/lab11avzN3248$ ./lab11avzN3248 -h ../test_data 0x414243
Входные данные: lab11abcNXXXXX [опции] каталог цель_поиска
Опции:
    -v, --version      Вывод версии программы и информации о программе (ФИО
                        исполнителя, номер группы, номер варианта лабораторной).
    -h, --help         Вывод справки по опциям.

Аргумент цель_поиска имеет формат 0xhh[hh*], где hh – две шестнадцатеричных
цифры (число должно начинаться с префикса 0x).
Пример: lab11abcNXXXXX /home 0xc0ffee
```

```
admin@Ubuntu:~/lab11avzN3248$ ./lab11avzN3248 --version ../test_data 0x414243
ОСП ЛР1.1
Версия: 1.0
Автор: Жук Анастасия Валерьевна
Группа: N3248
admin@Ubuntu:~/lab11avzN3248$ ./lab11avzN3248 --help ../test_data 0x414243
Входные данные: lab11abcNXXXXX [опции] каталог цель_поиска
Опции:
    -v, --version      Вывод версии программы и информации о программе (ФИО
                        исполнителя, номер группы, номер варианта лабораторной).
    -h, --help         Вывод справки по опциям.

Аргумент цель_поиска имеет формат 0xhh[hh*], где hh – две шестнадцатеричных
цифры (число должно начинаться с префикса 0x).
Пример: lab11abcNXXXXX /home 0xc0ffee
```

```

admin@Ubuntu:~/lab11avzN3248$ ./lab11avzN3248 ./ 0xhh
Ошибка: некорректное задание hex-строки
admin@Ubuntu:~/lab11avzN3248$ ./lab11avzN3248 ./ 0123
Ошибка: некорректные входные данные
admin@Ubuntu:~/lab11avzN3248$ ./lab11avzN3248 ./ OSP
Ошибка: некорректные входные данные
admin@Ubuntu:~/lab11avzN3248$ ./lab11avzN3248 -k ./ 0x414243
Ошибка: введена неподдерживаемая опция: 'k'
admin@Ubuntu:~/lab11avzN3248$ ./lab11avzN3248 -k -l ./ 0x414243
Ошибка: введена неподдерживаемая опция: 'k'
admin@Ubuntu:~/lab11avzN3248$ ./lab11avzN3248
Ошибка: отсутствуют необходимые аргументы
Входные данные: lab11abcNXXXXX [опции] каталог цель_поиска
Опции:
    -v, --version      Вывод версии программы и информации о программе (ФИО
                        исполнителя, номер группы, номер варианта лабораторной).
    -h, --help         Вывод справки по опциям.

Аргумент цель_поиска имеет формат 0xhh[hh*], где hh – две шестнадцатеричных
цифры (число должно начинаться с префикса 0x).
Пример: lab11abcNXXXXX /home 0xc0ffee
admin@Ubuntu:~/lab11avzN3248$ ./lab11avzN3248 ./
Ошибка: отсутствуют необходимые аргументы
Входные данные: lab11abcNXXXXX [опции] каталог цель_поиска
Опции:
    -v, --version      Вывод версии программы и информации о программе (ФИО
                        исполнителя, номер группы, номер варианта лабораторной).
    -h, --help         Вывод справки по опциям.

Аргумент цель_поиска имеет формат 0xhh[hh*], где hh – две шестнадцатеричных
цифры (число должно начинаться с префикса 0x).
Пример: lab11abcNXXXXX /home 0xc0ffee

```

```

admin@Ubuntu:~/lab11avzN3248$ ./lab11avzN3248 0x414243
Ошибка: отсутствуют необходимые аргументы
Входные данные: lab11abcNXXXXX [опции] каталог цель_поиска
Опции:
    -v, --version      Вывод версии программы и информации о программе (ФИО
                        исполнителя, номер группы, номер варианта лабораторной).
    -h, --help         Вывод справки по опциям.

Аргумент цель_поиска имеет формат 0xhh[hh*], где hh – две шестнадцатеричных
цифры (число должно начинаться с префикса 0x).
Пример: lab11abcNXXXXX /home 0xc0ffee

```

```
admin@Ubuntu:~/lab11avzN3248$ ./lab11avzN3248 ./ 0x414243
./exact_match.bin
admin@Ubuntu:~/lab11avzN3248$ ./lab11avzN3248 NULL 0x414243
Ошибка: путь к директории не может быть NULL или пустым
Входные данные: lab11abcNXXXXX [опции] каталог цель_поиска
Опции:
  -v, --version      Вывод версии программы и информации о программе (ФИО
                     исполнителя, номер группы, номер варианта лабораторной).
  -h, --help         Вывод справки по опциям.

Аргумент цель_поиска имеет формат 0xhh[hh*], где hh – две шестнадцатеричных
цифры (число должно начинаться с префикса 0x).
Пример: lab11abcNXXXXX /home 0xc0ffee
admin@Ubuntu:~/lab11avzN3248$ ./lab11avzN3248 "" 0x414243
Ошибка: путь к директории не может быть NULL или пустым
Входные данные: lab11abcNXXXXX [опции] каталог цель_поиска
Опции:
  -v, --version      Вывод версии программы и информации о программе (ФИО
                     исполнителя, номер группы, номер варианта лабораторной).
  -h, --help         Вывод справки по опциям.

Аргумент цель_поиска имеет формат 0xhh[hh*], где hh – две шестнадцатеричных
цифры (число должно начинаться с префикса 0x).
Пример: lab11abcNXXXXX /home 0xc0ffee
admin@Ubuntu:~/lab11avzN3248$ ./lab11avzN3248 ./ 0xhh
Ошибка: некорректное задание hex-строки
```

Исходный текст программы

Листинг 2 – Исходный текст программы

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <fts.h>
#include <errno.h>
#include <getopt.h>
#include <ctype.h>
#include <dirent.h>

#define VERSION "1.0"
#define AUTHOR "Жук Анастасия Валерьевна"
#define GROUP "N3248"
#define LAB_NAME "ОСП ЛР1.1"

// Структура для хранения байтовой последовательности
typedef struct {
    unsigned char *bytes;
    size_t length;
} ByteLine;

// Функция для преобразования строки с hex в байтовую последовательность
int hex_to_bytes(const char *str, ByteLine *line) {
    // Проверка входной строки
    if (str == NULL || line == NULL || strncmp(str, "0x", 2) != 0) {
        fprintf(stderr, "Ошибка: некорректные входные данные\n");
        return -1;
    }

    size_t len = strlen(str) - 2;
    if (len == 0 || len % 2 != 0) {
        fprintf(stderr, "Ошибка: некорректное задание hex-строки\n");
        return -1;
    }

    line->length = len / 2;
```

```

line->bytes = malloc(line->length);
if (line->bytes == NULL) {
    fprintf(stderr, "Ошибка: не удалось выделить память\n");
    return -1;
}

// Осуществляем преобразование
for (size_t i = 0; i < line->length; i++) {
    char byte_str[3] = {str[2 + i*2], str[2 + i*2 + 1], '\0'};
    char *endptr;
    long byte = strtol(byte_str, &endptr, 16);

    if (*endptr != '\0' || byte < 0 || byte > 255) {
        fprintf(stderr, "Ошибка: некорректное задание hex-строки\n");
        free(line->bytes);
        return -1;
    }

    line->bytes[i] = (unsigned char)byte;
}

return 0;
}

// Функция для проверки, содержит ли файл заданную последовательность
int line_in_file(const char *path, const ByteLine *line) {
    if (path == NULL || line == NULL || line->length == 0) {
        return 0;
    }

    FILE *file = fopen(path, "rb");
    if (file == NULL) {
        if (getenv("LAB11DEBUG") != NULL) {
            fprintf(stderr, "Ошибка: не удалось открыть файл %s: %s\n", path,
                strerror(errno));
        }
        return 0;
    }

    unsigned char *buffer = malloc(line->length);
    if (buffer == NULL) {
        fprintf(stderr, "Ошибка: не удалось выделить память\n");
    }

```

```

        fclose(file);
        return 0;
    }

    int found = 0;
    size_t offset = 0;

    while (fread(buffer, 1, line->length, file) == line->length) {
        if (memcmp(buffer, line->bytes, line->length) == 0) {
            found = 1;
            if (getenv("LAB11DEBUG") != NULL) {
                fprintf(stderr, "Нашлась последовательность в файле %s со смещением
%zu\n", path, offset);
            }
            break;
        }
        offset++;
        fseek(file, offset, SEEK_SET); // Перемещаем указатель на новую позицию
    }

    free(buffer);
    fclose(file);
    return found;
}

// Функция для обработки каталогов
void walk_function(const char *dir_path, const ByteLine *line) {
    DIR *dir = opendir(dir_path);
    if (dir == NULL) {
        fprintf(stderr, "Ошибка: не удалось открыть %s: %s\n", dir_path, strerror(errno));
        return;
    }
    closedir(dir);

    // Подготавливаем массив путей для обхода
    char *paths[] = {(char *)dir_path, NULL};

    // Открываем поток для обхода файловой системы
    FTS *fts_h = fts_open(paths, FTS_PHYSICAL | FTS_NOCHDIR, NULL);
    if (!fts_h) {
        fprintf(stderr, "Ошибка: не удалось открыть директорию %s: %s\n", dir_path,
strerror(errno));
    }
}

```

```

    return;
}

FTSENT *ent; // Указатель на информацию о текущем файле
while ((ent = fts_read(fts_h)) != NULL) {
    // Обрабатываем только обычные файлы
    if (ent->fts_info == FTS_F) {
        // Проверяем содержимое файла на совпадение
        if (line_in_file(ent->fts_path, line)) {
            // Выводим полный путь к найденному файлу
            printf("%s\n", ent->fts_path);
        }
    }
    // Обрабатываем ошибки доступа
    else if (ent->fts_info == FTS_DNR || ent->fts_info == FTS_ERR) {
        if (getenv("LAB11DEBUG") != NULL) {
            fprintf(stderr, "Ошибка: не удалось получить доступ к %s: %s\n", ent-
>fts_path, strerror(ent->fts_errno));
        }
    }
}

// Закрываем поток
fts_close(fts_h);
}

// Функция для вывода справки
void print_help() {
    printf("Входные данные: lab1 labcNXXXXXX [опции] каталог цель_поиска\n");
    printf("Опции:\n");
    printf(" -v, --version    Вывод версии программы и информации о программе\n\n");
    printf("                  исполнителя, номер группы, номер варианта лабораторной).\n");
    printf(" -h, --help        Вывод справки по опциям.\n");
    printf("\n");
    printf("Аргумент цель_поиска имеет формат 0xhh[hh*], где hh — две\n\n");
    printf("шестнадцатеричных\n");
    printf("цифры (число должно начинаться с префикса 0x).\n");
    printf("Пример: lab11abcNXXXXXX /home 0xc0ffee\n");
}

// Функция для вывода версии

```

```

void print_version() {
    printf("%s\n", LAB_NAME);
    printf("Версия: %s\n", VERSION);
    printf("Автор: %s\n", AUTHOR);
    printf("Группа: %s\n", GROUP);
}

int main(int argc, char *argv[]) {
    static struct option long_options[] = {
        {"help", no_argument, 0, 'h'},
        {"version", no_argument, 0, 'v'},
        {0, 0, 0, 0}
    };

    int opt;
    opterr = 0;
    while ((opt = getopt_long(argc, argv, "hv", long_options, NULL)) != -1) {
        switch (opt) {
            case 'h':
                print_help();
                return 0;
            case 'v':
                print_version();
                return 0;
            default:
                fprintf(stderr, "Ошибка: введена неподдерживаемая опция: '%c'\n", optopt);
                return 1;
        }
    }

    if (argc - optind < 2) {
        fprintf(stderr, "Ошибка: отсутствуют необходимые аргументы\n");
        print_help();
        return 1;
    }

    const char *directory = argv[optind];
    const char *search_bytes = argv[optind + 1];

    // Проверка, что directory не NULL и не пустая строка
    if (directory == NULL || strcmp(directory, "NULL") == 0 || directory[0] == '\0') {
        fprintf(stderr, "Ошибка: путь к директории не может быть NULL или пустым\n");
    }
}

```

```
    print_help();
    return 1;
}

ByteLine line;
if (hex_to_bytes(search_bytes, &line)) {
    return 1;
}

if (getenv("LAB11DEBUG") != NULL) {
    fprintf(stderr, "Debug: осуществляется поиск последовательности: ");
    for (size_t i = 0; i < line.length; i++) {
        fprintf(stderr, "0x%02x ", line.bytes[i]);
    }
    fprintf(stderr, "\n");
}

walk_function(directory, &line);

free(line.bytes);
return 0;
}
```